

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



Code of Conduct:

*I _____ Soorya S / 192311060_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Answer

1. Testing as an Engineering Activity in a Banking Application

Question

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer

Testing as an engineering activity means testing should be a systematic, planned, measurable, and repeatable process rather than random execution of test cases.

The banking team should first define a **test strategy and test plan** based on requirements and risks. Test cases should be designed systematically using techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

Instead of ad-hoc testing, testers can use **requirement traceability matrices (RTM)** to ensure every requirement is tested.

Testing tools can be introduced for different activities. **Selenium** can automate web testing, **JMeter** can perform load testing, and tools such as **Jira** can manage and track defects.

Standards such as **IEEE-based test documentation practices** and organizational coding/testing standards can be followed. Testing should also be integrated with CI/CD so that automated tests execute whenever new code is delivered.

Practical Example

For an online fund-transfer feature, boundary values such as ₹1, ₹10,000, ₹10,000.01 and invalid amounts can be tested. Decision tables can verify combinations of sufficient balance, valid account, OTP verification, and transaction limits.

This approach detects defects earlier and provides measurable coverage. Therefore, replacing random testing with **risk-based, systematic, automated, and documented testing** improves reliability and reduces production failures.

2. Structured Testing Process for an ATM System

Question

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer

A structured testing process should be implemented throughout the **Software Development Life Cycle (SDLC)** rather than only after coding.

1. Requirement Analysis

Testers review requirements such as:

- Valid ATM card
- Correct PIN
- Available balance
- Daily withdrawal limit
- Cash availability

Ambiguous requirements are identified early.

2. Test Planning

A test plan defines scope, resources, tools, risks, test environment, and schedule.

3. Test Case Design

Test cases are designed using **equivalence partitioning, boundary value analysis, and decision tables**.

4. Test Environment Setup

The ATM simulator, database, card reader, and banking server are configured.

5. Test Execution

Testers execute test cases and record actual results against expected results.

6. Defect Reporting and Retesting

Failures are reported with steps, expected result, actual result, severity, and evidence. After fixing, the defect is retested.

7. Regression Testing

Existing ATM functions are tested again after changes.

ATM Withdrawal Example

For a withdrawal of ₹10,000:

1. Insert valid card.
2. Enter correct PIN.
3. Select withdrawal.
4. Enter ₹10,000.
5. System checks balance and withdrawal limit.
6. Cash is dispensed.
7. Account balance is updated.
8. Receipt is generated.

Invalid PIN, insufficient balance, exceeded daily limit, and unavailable cash should also be tested.

A consistent process reduces missed scenarios and duplicate efforts. It results in **fewer defects, predictable testing, better traceability, and reliable ATM transactions.**

3. Error, Defect, and Failure in a Web Application

Question

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer

The terms **error, defect, and failure** represent different stages of a problem.

Error

An **error** is a human mistake made during requirements, design, coding, or testing.

Example: A developer misunderstands the requirement and writes code allowing users to transfer ₹1,00,000 instead of the permitted ₹50,000.

Defect

A **defect** is an incorrect condition in the software caused by an error.

Example: The transaction validation code contains an incorrect limit of ₹1,00,000.

Failure

A **failure** occurs when the software executes the defective code and produces an incorrect result.

Example: A customer successfully transfers ₹80,000 even though the maximum allowed amount is ₹50,000.

The relationship can be represented as:

Human Error → Software Defect → Execution → Failure

Login Example

If a developer incorrectly implements password validation, the incorrect validation logic is a **defect**. When a user with an invalid password is allowed to log in, it becomes a **failure**.

Improper understanding can result in unclear defect reports such as *"Login not working."* This makes debugging difficult.

A good defect report should contain:

- Defect ID
- Module
- Preconditions
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshot/log
- Environment

Clear terminology and structured reports improve communication between testers and developers and make defect resolution faster.

4. Defect Clustering and Test Prioritization

Question

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer

Defect clustering is a testing principle stating that a small number of modules often contain a large percentage of defects.

Testers should analyze historical defect data and identify high-risk modules. These modules should receive more testing effort.

Other important testing principles include:

- Testing shows the presence of defects, not their absence.
- Exhaustive testing is impossible.
- Early testing saves time and cost.
- Testing is context dependent.
- Repeating the same tests may become less effective.

Test Prioritization

Test cases can be prioritized based on:

1. Business importance
2. Failure probability
3. Historical defects
4. Security risk
5. Customer impact
6. Complexity of the module

For example:

Test Area	Risk	Priority
Payment	Very High	P1
Login	High	P1
Product Search	Medium	P2
UI Theme	Low	P3

Real-Time Example

In an e-commerce application, historical data may show that **payment and checkout modules contain 60% of defects**. Therefore, testers should allocate more test cases and automation to these modules.

This approach avoids spending equal effort on low-risk and high-risk areas. Defect clustering helps testers focus resources where failures are more likely, resulting in **faster defect detection, reduced testing time, and improved product quality**.

5. Defect Classification and Defect Repository

Question

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer

A **defect repository** is a centralized collection of previously identified defects and their associated information. It can be used not only for tracking but also for preventing future defects.

Origin of Defects

Defects can originate during:

- Requirement analysis
- Design
- Coding
- Integration
- Configuration
- Testing

Defect Classification

Defects may be classified as:

- Functional defects
- Performance defects
- Security defects
- UI defects
- Database defects
- Integration defects
- Compatibility defects

They can also be classified by severity and priority.

Using Repository Data

Suppose previous releases show repeated defects in payment calculations. Testers should analyze these historical defects and create additional regression test cases around payment calculations.

The repository can provide information such as:

- Frequently defective modules
- Repeated defect types
- Root causes
- Defect severity
- Defect frequency
- Release-wise defect trends

Prevention

The organization should:

1. Perform root-cause analysis.
2. Add regression tests for recurring defects.
3. Update test checklists.
4. Conduct code reviews.
5. Improve requirement validation.
6. Automate frequently repeated tests.

For example, if a tax calculation defect occurs in three releases, a permanent regression test should be created for different tax slabs.

Thus, a defect repository becomes a **knowledge base for future testing**, reducing recurring defects and improving software quality.

6. Testing Principles for a Mobile Banking Fund Transfer

Question

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer

Frequent crashes during fund transfer indicate that the application may have defects related to input validation, network handling, memory usage, concurrency, or backend integration.

A systematic testing approach should begin with **requirement and risk analysis**.

Testing Approach

Testers should check:

- Valid and invalid account numbers
- Different transfer amounts
- Insufficient balance
- Network interruption
- Server timeout
- Multiple simultaneous transactions
- OTP expiration
- Session timeout
- App backgrounding
- Different mobile devices and OS versions

Root-Cause Analysis

Logs and crash reports should be collected. For example, if crashes occur whenever the network changes from Wi-Fi to mobile data, testers can reproduce the condition and provide logs to developers.

Testing Techniques

- Boundary Value Analysis
- Equivalence Partitioning
- State Transition Testing
- Negative Testing
- Compatibility Testing
- Performance Testing
- Security Testing

Example

Suppose a transfer is interrupted after money is deducted but before confirmation. Testing should verify whether the transaction is rolled back or safely marked as pending.

Automated regression tests should be executed after every major change.

Systematic testing identifies failures under normal, abnormal, and edge conditions. Continuous monitoring, crash analytics, automation, and regression testing can prevent similar failures in future releases.

7. Testing vs Debugging

Question

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer

Testing and debugging are related but different activities.

Testing

Testing is the process of executing or evaluating software to **identify defects and determine whether requirements are satisfied**.

The tester generally answers:

"Is the software behaving correctly?"

Debugging

Debugging is the process of **locating, analyzing, and fixing the cause of a defect**.

The developer generally answers:

"Why did the software behave incorrectly, and how can I fix it?"

Example

Suppose an ATM deducts ₹10,000 but displays ₹5,000.

- Tester identifies the mismatch → **Testing**
- Developer investigates calculation/database code → **Debugging**
- Developer fixes the code → **Debugging**
- Tester executes the test again → **Retesting**
- Tester checks related functions → **Regression Testing**

If testers attempt to fix code themselves without proper responsibility, or developers only perform random testing, the process becomes inefficient.

Structured Approach

Requirement → Test Design → Test Execution → Defect Report → Debugging → Fix → Retesting → Regression Testing

Defect reports should contain reproducible steps, expected/actual results, logs, severity, and environment details.

Separating testing from debugging while maintaining good collaboration ensures that defects are detected systematically and fixed efficiently.

8. Cross-Browser Testing for an E-Commerce Platform

Question

An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer

Cross-browser testing verifies whether a web application behaves consistently across different browsers, operating systems, and screen sizes.

Defects may occur because browsers use different rendering engines, JavaScript implementations, CSS support, or security policies.

Testing Strategy

Test the application on:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Different operating systems and screen resolutions should also be considered.

Important functions include:

- Login
- Product search
- Cart
- Checkout
- Payment
- Account management

Example

Suppose the **Add to Cart** button works correctly in Chrome but becomes unresponsive in Firefox. The tester should reproduce the problem, record browser and OS versions, capture screenshots/logs, and report the defect.

Tools

- Selenium
- Playwright
- BrowserStack
- LambdaTest
- Developer Tools

Automation can execute the same test cases across multiple browsers.

A browser compatibility matrix can be created to identify required combinations.

Systematic cross-browser testing increases coverage, identifies compatibility defects earlier, and provides a consistent customer experience across platforms.

9. Defect Classification in Payroll Salary Calculation

Question

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer

Incorrect salary calculations are primarily **functional or logical defects** because the system does not implement the required business rules correctly.

Possible defect categories include:

- Calculation/logic defects
- Requirement defects
- Database defects
- Integration defects
- Boundary defects
- Data validation defects

Example

Suppose:

Basic Salary = ₹30,000

Allowance = ₹5,000

Deduction = ₹3,000

Expected salary:

$$\text{₹30,000} + \text{₹5,000} - \text{₹3,000} = \text{₹32,000}$$

If the program incorrectly subtracts the allowance, the result becomes incorrect. This is a **logic defect**.

Testing Techniques

Testers should use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- Data-driven testing
- Unit testing
- Integration testing
- Regression testing

Boundary cases such as minimum salary, maximum salary, zero allowance, maximum deductions, and tax thresholds should be tested.

Developers can prevent such defects through code reviews, unit tests, static analysis, and automated regression suites.

Understanding defect classification helps testers select appropriate techniques. As a result, payroll calculations become more accurate, reliable, and compliant with business rules.

10. Replacing Random Testing with Structured Test Design

Question

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer

Random testing without planning can miss important requirements and edge cases. Testing should therefore be treated as an **engineering activity** with systematic test design.

The redesigned process should be:

Requirement Analysis → Risk Analysis → Test Design → Test Data → Test Execution → Defect Reporting → Retesting → Regression Testing

Importance of Test Case Design

A well-designed test case contains:

- Test ID
- Requirement
- Preconditions
- Input
- Steps
- Expected result
- Actual result
- Pass/Fail status

Suitable Techniques

Equivalence Partitioning: Divides inputs into valid and invalid groups.

Boundary Value Analysis: Tests values at and around boundaries.

Decision Table: Tests combinations of conditions.

State Transition Testing: Tests changes between system states.

Use Case Testing: Tests complete user workflows.

Example

For a password field allowing 8–16 characters, test:

7, 8, 9, 15, 16, 17 characters.

Random testing may test only 10 characters and miss boundary defects.

Structured test design provides better requirement coverage, reduces duplicate testing, detects edge-case defects, and makes testing measurable and repeatable.

11. Data Inconsistency in a Healthcare Application

Question

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer

Data inconsistency occurs when different modules display or store different values for the same information. In a healthcare application, this can be a serious quality issue.

Possible Causes

Defects may originate from:

- Incorrect database queries
- Improper data mapping
- Synchronization errors
- API defects
- Incorrect validation
- Transaction failures
- Requirement misunderstandings

Example

Suppose a patient's blood group is recorded as **O+** in the registration module but displayed as **A+** in the medical-record module. This indicates a possible data synchronization or mapping defect.

Testing Approach

Testers should perform:

- Database testing
- Integration testing
- API testing
- Data validation
- End-to-end testing
- Regression testing

The same patient data should be followed across all modules.

A traceability matrix can map requirements to test cases.

Defect Reporting

Each defect should include:

- Patient/test data identifier
- Module
- Steps to reproduce
- Expected value
- Actual value
- Database/API evidence
- Severity
- Environment

Because healthcare data is sensitive, test data should be properly controlled and anonymized.

Systematic integration and database testing ensures consistency and improves data integrity across the application.

12. Repeated Defects and Defect Repository

Question

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer

A defect repository should not be treated only as a list of bugs. It should function as a **historical knowledge base** for improving future testing.

Every defect should contain:

- Defect ID
- Module
- Defect type
- Root cause
- Severity
- Priority
- Release
- Status
- Resolution
- Related test case

Identifying Patterns

Suppose repository data shows:

Module	Defects
Payment	40
Login	15
Search	5

The payment module is clearly a high-risk area.

If 20 payment defects were caused by incorrect validation, testers should create additional validation and boundary test cases.

Preventive Actions

1. Perform root-cause analysis.
2. Add failed scenarios to regression suites.
3. Review requirements.
4. Conduct code reviews.
5. Increase testing in high-risk modules.
6. Automate recurring tests.
7. Track defect trends across releases.

For example, if an incorrect tax calculation occurs repeatedly, a permanent automated regression test should be created.

Using repository information transforms past failures into future test knowledge. This reduces repeated defects and improves testing effectiveness and software quality.

13. Web Learning System Failing During Peak Usage

Question

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer

The failure during peak usage indicates inadequate **performance and load testing**.

Functional testing alone cannot determine whether the application can handle thousands of simultaneous users.

Testing Gaps

The team may have failed to test:

- Concurrent users
- Peak traffic
- Large database requests
- Server capacity
- Response time
- Network delays
- Resource utilization
- Database bottlenecks

Test Planning

The test plan should define:

- Expected user load
- Peak load
- Response-time requirements
- Hardware/software environment
- Performance metrics
- Acceptance criteria

Testing Types

Load Testing: Tests expected user load.

Stress Testing: Tests beyond normal capacity.

Spike Testing: Tests sudden increases in traffic.

Endurance Testing: Tests prolonged usage.

Example

If 10,000 students are expected to access an online examination simultaneously, testing only with 100 users is insufficient.

Tools such as **JMeter** or **LoadRunner** can simulate large numbers of users.

Performance monitoring should measure CPU, memory, database response, throughput, and response time.

Better planning and coverage allow bottlenecks to be identified before deployment. This improves scalability, availability, and reliability during peak usage.

14. Early Involvement of Testers in SDLC

Question

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer

Testers should participate from the **requirement analysis stage**, not only after development is completed. This is called **early testing or shift-left testing**.

Tester Role Across SDLC

Requirement Phase: Review requirements for ambiguity and testability.

Design Phase: Identify integration and usability risks.

Development Phase: Review unit tests and participate in code reviews.

Testing Phase: Execute functional, integration, system, and regression tests.

Deployment: Validate release readiness.

Maintenance: Perform regression and production verification.

Practical Example

Suppose a banking requirement says:

"Users can transfer money within the permitted limit."

A tester can immediately ask:

- What is the minimum amount?
- What is the maximum amount?
- What happens when the limit is exceeded?
- What happens during network failure?
- Is OTP mandatory?

These questions can identify missing requirements before coding begins.

Early defect detection is cheaper than fixing defects after deployment. Collaboration through Agile ceremonies, requirement reviews, design reviews, and continuous testing improves communication.

Therefore, involving testers early reduces rework, prevents requirement defects, and improves overall software quality.

15. Incomplete Test Coverage

Question

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer

Incomplete test coverage means that only some possible conditions, inputs, paths, or requirements have been tested.

A system may pass normal scenarios but fail under boundary or invalid conditions.

Identifying Gaps

Testers should analyze:

- Requirement coverage
- Functional coverage
- Code coverage
- Boundary conditions
- Negative scenarios
- User workflows
- Integration points

A **Requirement Traceability Matrix (RTM)** can ensure every requirement has corresponding test cases.

Example

For a bank withdrawal limit of ₹20,000, testing only ₹5,000 is insufficient.

Important tests include:

- ₹1
- ₹19,999
- ₹20,000
- ₹20,001
- Negative value
- Insufficient balance

Techniques

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Pairwise Testing
- Risk-based Testing

Automation can execute large regression suites repeatedly.

Proper coverage does not mean testing every possible input, which is usually impossible. Instead, testers should select representative and high-risk conditions systematically.

Improved coverage increases confidence, detects hidden defects, and improves system reliability.

16. Improving Defect Reporting in a Logistics Application

Question

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer

Poor defect reports make it difficult for developers to understand and reproduce problems. A defect report should be **clear, complete, reproducible, and objective**.

Recommended Defect Report

A structured report should include:

1. Defect ID
2. Title
3. Module
4. Environment
5. Preconditions
6. Steps to reproduce
7. Test data
8. Expected result
9. Actual result
10. Severity
11. Priority
12. Screenshots/logs
13. Reproducibility
14. Status

Example

Poor report:

"Delivery tracking is not working."

Better report:

"Tracking status does not update after a delivery scan."

Then provide exact steps, test shipment ID, expected status, actual status, timestamp, browser/device, and logs.

Severity vs Priority

Severity represents the impact of the defect.

Priority represents how urgently it should be fixed.

For example, a payment failure may be **High Severity and High Priority**.

Using standardized defect templates and tools such as Jira improves communication between testers, developers, and managers.

Clear reporting reduces investigation time, prevents duplicate defects, improves tracking, and speeds up defect resolution.

17. Process-Based Testing for Consistency

Question

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer

Process-based testing uses defined and repeatable procedures for planning, designing, executing, reporting, and closing testing activities.

The organization should establish a standard testing lifecycle:

Requirement Analysis → Test Planning → Test Design → Environment Setup → Execution → Defect Management → Retesting → Regression → Test Closure

Standardization

Common standards should define:

- Test case format
- Naming conventions
- Defect reporting format
- Severity levels
- Entry/exit criteria
- Review procedures
- Automation guidelines

Example

Suppose the login, payment, and shopping-cart teams currently use different test case formats. A common template can be introduced for all modules.

Every module must include:

- Positive tests
- Negative tests
- Boundary tests
- Security checks
- Regression tests

A common checklist and centralized test management system can improve consistency.

Regular test-case reviews and audits can verify compliance.

Standardized processes make testing measurable and repeatable. They reduce missed scenarios, improve communication, simplify maintenance, and produce consistent quality across modules.

18. Boundary Testing for Untested Edge Cases

Question

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer

Edge cases are often missed because testers concentrate mainly on normal inputs. **Boundary Value Analysis (BVA)** is an effective technique for detecting such defects.

BVA focuses on values at and around the limits of an input range.

Example

Suppose an application allows users aged **18–60**.

Important test values are:

- 17 → Invalid
- 18 → Valid
- 19 → Valid

- 59 → Valid
- 60 → Valid
- 61 → Invalid

If developers incorrectly implement the condition as `age > 18`, a valid user aged 18 may be rejected.

Why Edge Cases Are Missed

- Incomplete requirements
- Poor test design
- Time constraints
- Over-reliance on normal scenarios
- Lack of historical defect analysis

Improvements

Testers should combine:

- Boundary Value Analysis
- Equivalence Partitioning
- Decision Tables
- Negative Testing
- Risk-based Testing

Requirements should clearly specify minimum and maximum values.

Edge cases found in production should also be added to the regression suite.

Systematic boundary testing improves defect detection and prevents failures caused by incorrect limits or conditions.

19. Defect Repository and Documentation

Question

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer

A **defect repository** is a centralized system that stores information about defects throughout their lifecycle.

A properly documented defect should include:

- Unique defect ID

- Title
- Description
- Module
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Reporter
- Assignee
- Status
- Resolution
- Evidence

Example

Instead of writing:

"Payment failed."

The report should explain the exact transaction conditions, steps, expected payment status, actual status, error message, and supporting logs.

Defect Lifecycle

New → Assigned → Open → Fixed → Retest → Verified → Closed

If the defect still exists:

Retest → Reopened

A structured repository allows managers to analyze:

- Defects by module
- Defects by severity
- Recurring defects
- Defect trends
- Average resolution time
- Root causes

For example, if database defects repeatedly occur in the payment module, additional database testing and code reviews can be introduced.

Using standardized templates, mandatory fields, controlled severity definitions, and regular repository analysis improves traceability and prevents repeated defects.

20. Redesigning Testing for a Banking Transaction System

Question

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Answer

Repeated transaction failures indicate that testing is not sufficiently systematic or risk-oriented. A structured testing strategy should be introduced.

1. Requirement Analysis

Identify transaction rules such as:

- Valid account
- Sufficient balance
- Transaction limits
- OTP requirements
- Duplicate transaction prevention
- Transaction timeout

2. Risk-Based Test Planning

High-risk functions such as payment, fund transfer, and balance updates receive higher testing priority.

3. Test Design

Use:

- Equivalence Partitioning
- Boundary Value Analysis

- Decision Tables
- State Transition Testing
- Negative Testing
- Use Case Testing

Example

For a ₹50,000 transaction limit:

Input	Expected Result
₹1	Accept
₹50,000	Accept
₹50,001	Reject
Negative amount	Reject
Insufficient balance	Reject

4. Integration Testing

Verify communication between:

Mobile/Web Application → API → Transaction Server → Database → Banking System

5. Failure Scenarios

Test:

- Network failure
- Server timeout
- Duplicate requests
- Session timeout
- Database failure
- OTP expiration

6. Automation and Regression

Automated regression tests should run whenever transaction-related code changes.

7. Defect Repository

Previous transaction defects should be analyzed and converted into permanent regression tests.

A structured process improves coverage, detects defects earlier, reduces repeated failures, and increases confidence in transaction reliability. For a banking system, combining **risk-based testing, automation, boundary testing, integration testing, negative testing, and regression testing** provides much stronger defect detection than ad-hoc testing.
